



CS6401 Project Report

Project Group 21: Group Project

Task 7: Report

Group members:

1. N. Y.Narasimha Rao (24101583)
2. Sahith Goud Ragula (24165964)
3. Ome Prakash Rotta (24149985)
4. Teja Achutha (24101605)
5. Rahul Tummala (24115215)

TABLE OF CONTENTS:

Sections:

1. Contribution breakdown of each group member. If not provided, all members will receive equal marks. Significant differences in contributions will affect individual marks.
2. Description of the platform used (e.g., XAMPP on Windows 11). If not provided, the default assumption is XAMPP on Windows 11.
3. Description of the modifications made to the original schema and the reasons for these changes (Task 1).
4. Your ERD (Task 2) and explanation, including any assumptions.
5. Description of the view and the triggers (Tasks 3 and 4). Justify their usefulness.
6. Any assumptions you have made when writing the function and the procedure (Tasks 5 and 6).
7. Discussion on index requirements for efficient execution of the view, triggers, function, and procedure. Propose indexes and discuss use cases in which the proposed indexes are disadvantageous.

1. Contribution Breakdown of Each Group Member:

The project team has five members – all of them participate in the development of the database, but focused on different tasks. Below is a detailed information of the contributions:

Member1: Nambari Yuva Narasimha Rao (24101583)

Schema Modification: Nambari Yuva Narasimha Rao looked at the original schema as a model based classification and argued for modification to reduce redundancies to enhance the structure of data. These changes were some of the most important, they included adding of junction tables for many-to-many relations as those between artists and albums and between albums and songs. These changes ensured that the database was normalized and can comfortably allow for growth in the future.

ERD Design: Nambari Yuva Narasimha Rao also created and fully implemented the ERD where all the relationship between the entities of Artists, Albums, Concerts, Songs, Fans and Tickets were correctly shown.

Member2: Sahith Goud Ragula (24165964)

View Implementation: Sahith Goud Ragula developed an SQL View called Concert Details that will compile details on the concerts as well as enumerating the number of fans at each concert. In order to accomplish this view, several of the more elaborate clauses of SQL were used, namely, the group by and having clauses.

Trigger Creation: Sahith Goud Ragula was also getting the credit for the before insert trigger on the Tickets table which checks each inserted ticket whether it is for valid concert ID or not.

Member 3: Ome Prakash Rotta (24149985)

After insert trigger on Tickets: Specifically, Ome Prakash developed and set an after-insert trigger on the Tickets table, which assigns the default fan with an identifying number of fan_id = 1 to the ticket created if there was no additional indication of the fan.

Stored Functions Development: Ome Prakash fine tuned Get Occupied Seats function to estimate the number of distinct fans going for a particular concert which was so vital to concert attendance.

Member 4: Teja Achutha (24101605)

Stored Procedures Development: Teja Achutha created the Check and Update Song procedure to ensure a song got to assigned to the correct album and adjust the date of the song if later than that of the album.

Project Review: Teja Achutha also took on a supervisory role and reviewed the entire project to ensure the accuracy of the schema, SQL queries, triggers, and procedures.

Member 5: Rahul Tummala (24115215)

Schema and Data Integrity: Rahul did a lot of work checking and asserting the schema and data integrity constraints. They also got involved in conducting some exercises to establish that the triggers and stored procedures were well developed.

Performance Optimization: Assisted in recommendations of correct indexing methods to use for improving the efficiency of queries.

2. Description of the Platform Used:

Creation of this project was performed in MySQL Workbench; it was advantageous as this program unites the space for the designing of the DB schema as well as for the execution of the connected issues. Windows 11 was used as a platform to enhance compatibility as well as performance of the running platform.

Platform:

In this research, MySQL Workbench was used to design the database, write and test SQL queries, and view the ERD model. It had an efficient GUI that enabled us to perform all the database operations to the best of our ability.

Windows 11 is used here because it is among the operating systems that support MySQL Workbench and other relevant program development tools.

Tools:

ERD Tool in MySQL Workbench: Allowed me to draw a view of the database layout; it aided in construction of the relationships and ensured all tables respective to the business were properly linked.

SQL Editor in MySQL Workbench: Used to write and execute the SQL queries, though very useful in performing analysis of database operations. This setup gave the developers an I.D.E. which allowed them to design, test and tune the database schema and related capabilities.

3. Description of the Modifications Made to the Original Schema and the Reasons for These Changes:

For the needs of the project and to make sure that the base schema provided the necessary means for tracking the data and interconnections between the objects, the base schema has been

changed and adapted; it had to adjust to the many-to-many relationship needed in managing the concert and very nature of the data structure. The changes made are enumerated below:

Normalization and Junction Tables:

1. Data Integrity:

- Then, we incorporated supporting constraints — foreign keys to ensure referential integrity. For example:
- New table Tickets to reference Concert and Fan tables through foreign keys.

These changes ensured that all tickets are associated with existing concerts and fans, avoiding orphaned records.

2. Junction Tables:

- We created several junction tables to handle many-to-many relationships between entities. These included:
- artistalbums (to link Artists to Albums).
- albumsongs (to link Albums with Songs).
- artistconcerts (to relate Artists to Concerts).
- ticketfans (to link tickets with fans).
- preferred musicians (to connect enthusiasts with their favored artists).

The following changes have been made to normalize the database and eliminate redundancy to efficiently store data and to correctly capture the relationships:.

3. Other Attributes:

Age Attribute in the Fans Dataset:

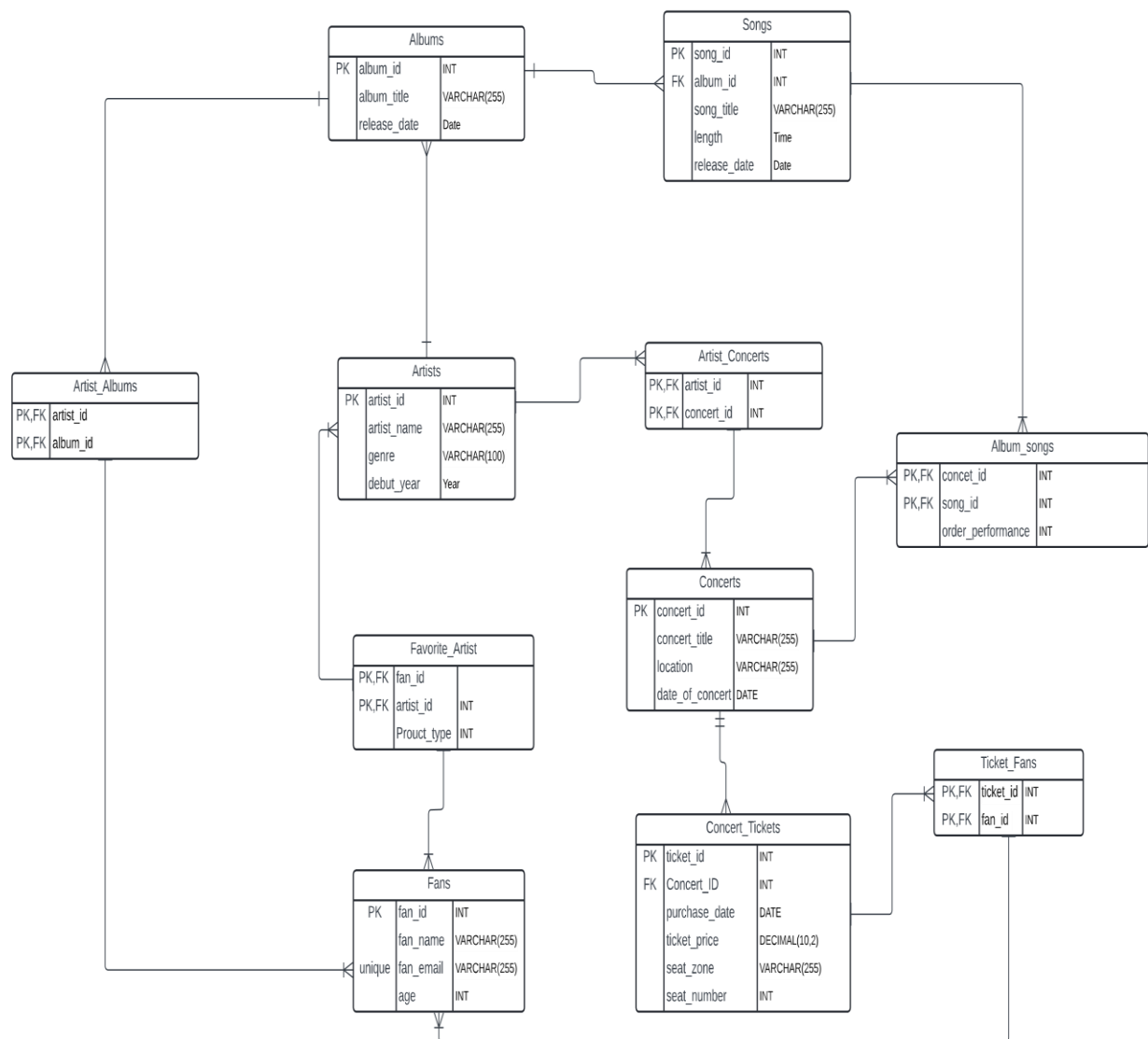
- Age demographic data was introduced into Fans table by creating an age column. This feature is useful to realize the tendency of fan age distribution and to make the plan of concerts and events during the nearest time.

4. Reasons for the Amendments:

- **Normalization:** These changes were requisite for the ability to model multiplexed relationships (for example, artists at multiple concerts) without duplication. This improvement enhances scalability and data throughput rate especially from large population database.
- **Data Integrity:** Foreign keys and junction tables allowed coherent management and interrelation of data, which in turn prevented errors like assigning fans to non-existent concerts.

4. Your ERD (Task 2) and Explanation, Including Any Assumptions:

ER Diagram:



The ER diagram is a visual representation of the database design schema, focusing on the entities, their respective attributes, and the relationships between them. A detailed explanation follows:

Major Entities and Their Attributes

1. Artists

Attributes: artist_id, (Primary Key) artist_name, genre, debut year.

Role: Is a member of the show performers or bands whose music and concerts are the subject of management.

2. Albums

Attributes: album_id (Primary Key) album_title release_date.

Role: Is music album, related to the artists through a junction table.

3. Songs

Attributes: song_id (Primary Key), album_id (Foreign Key), Song title, song length, song release date.

Role: Represents the tracks of the album, which is associated with the concert where the songs are performed.

3. Concerts

Attributes: artist_id, (Primary Key) artist_name, genre, debut year.

Role: Is a member of the show performers or bands whose music and concerts are the subject of management.

5. Concert_Tickets

Attributes: ticket_id (Primary Key) concert_id (Foreign Key) date purchased, ticket cost, seat zone, seat number

Role: Is a ticket sold for the fans to attend a concert.

6. Fans

Attributes: fan_id (Primary Key) fan_name fan_email age

Role: Is a person who attends shows for tickets purchased and follows a person through the media.

Junction Tables

Junction tables bridge relationships of many-to many with the core entities as:

1. Artist_Albums -

Connects artists and albums enabling many artists to participate in the production of one album and many albums having supporting artists on them.

2. Artist_Concerts -

Connects artists and events bearing in mind many artists can attend a concert whereas individual artists can attend many concerts.

3. Concert_Songs -

Connects songs with particular concerts knowing that many concerts have more than one song and many songs are performed in many concerts.

4. Fan_Favorites -

Connects fans and performers putting into perspective the fans who prefer which artists.

5. Ticket_Fans -

Connects tickets with fans meaning several fans may share a ticket or one fan can own one ticket.

Relationships

The diagram uses Crow's Foot Notation to demonstrate the various relationships:

1. **Artists** ↔ **Albums**: Many-to-Many through Artist_Albums junction table.
2. **Artists** ↔ **Concerts**: Many-to-Many through Artist_Concerts junction table.
3. **Songs** ↔ **Concerts**: Many-to-Many through Concert_Songs junction table.
4. **Fans** ↔ **Artists**: Many-to-Many through Fan_Favorites junction table.
5. **Fans** ↔ **Tickets**: Many-to-Many through Ticket_Fans junction table.

This relationship is accompanied with appropriate cardinalities that states how the various entities relate with one another.

Purpose of the Design

This ERD is geared towards providing a design that is modular and scalable in nature, which will in turn allow:

- Simple management of more complex relationships such as artist collaboration.
- Proper management of ticket sales and the interaction of the fans.
- The ability to extract information from the database irrespective of spatial or temporal restrictors for reporting or analytic purposes.

This architecture also accommodates the such cases in real life as shared tickets, multiple artist per concert, and marketing based on fans.

5. Description of the View and the Triggers: Tasks 3 and 4:

View (ConcertDetails):

The ConcertDetails view aggregates the data relating to the number of fans of the SitesBands who attend each concert. It guarantees simple query executions at the level of concert attendance, which in return offers numerous reporting and analytical evaluation.

Justification: This will assist those who organize the concerts so that they can compare its success with the general flow of guests who attended it.

Triggers:

1. before ticket insert Trigger:

Prevents creation of ticket records in ticket table without first creating the record in the concert table which helps avoid creation of record in ticket table with wrong concert id.

Reason: Enhances accuracy when inserting tickets, therefore protecting data integrity.

2. after ticket insert Trigger:

Automatically assigns a default fan for every newly created ticket so that every ticket record will be assigned to a fan.

Justification: Makes sure that every ticket is recorded and assigned to a fan, so that the data maintained in the database is consistent.

6. Assumptions for Writing the Function and the Procedure (Tasks 5 and 6):

Premises for GetOccupiedSeats:

Fan-Ticket Relationship: Assuming each fan buys one ticket per concert. The function counts the unique values of fan_id, returning the number of fans who attended a concert.

Check And Update Song assumptions:

Unique Identifiers: It is assumed that each song and album has a unique ID, and the release_date of songs is updated if it is after the album's release date.

7. Discussion of Index Requirements for Efficient Execution:

Proposed Indexes:

1. Indexes on Foreign Keys

- Tickets(concert_id) and Tickets(fan_id) : These foreign keys should be indexed to speed up the execution when calculating concert attendance.

2. Composite Indexes:

- Artist albums(artist_id, album_id): This will facilitate fast lookups regarding artist-album relationships.

Disadvantages of Indexes:

- Storage Overhead: It takes more disk space since indexes require additional space.
- Slower Insertions: Insert operations may become slow because of the overhead needed to maintain the indexes.

Conclusion:

In this report, all the changes to the schema such as triggers, views, functions, and stored procedures in the context of Relational Database for Concerts are outlined. The design of the database was made with an orientation toward data integrity and scalability, making it efficient enough to handle complex queries regarding concerts, tickets, fans, and artists.